HOTELPRO

Full-Stack Hotel Management System

Project Documentation

Project overview
A web-based hotel booking and property management application for guest reservations, hotel operations, and administration. Built with React, Express, and MongoDB/Mongoose.

Project	HotelPro
Repository	Yash-me1987/HotelPro
Document version	1.0
Prepared	October 2026

Table of Contents

Abstract

1. Introduction

2. Requirements

3. Software and Hardware Requirements

4. System Architecture

5. Application Modules

6. Data Design

7. API Reference

8. Installation and Operation

9. Testing

10. Implementation Notes and Production Readiness

11. Conclusion

Appendix A. Project Layout

HOTELPRO / SYSTEM OVERVIEW

Abstract

HotelPro is a web-based hotel booking and property management application. It combines a React client with an Express API and a MongoDB data layer. The application brings together guest-facing room discovery and reservation screens with staff workflows for arrivals, housekeeping, and service requests, plus administrative inventory and reporting screens.

The project is designed as a luxury-hotel experience, using a Tuscan olive and pale-yellow visual theme. Its backend supports MongoDB persistence and can fall back to in-memory sample data when MongoDB is unavailable. Payment, email, and demo-mode behavior depend on configuration; the development fallback behavior described in this document is not a substitute for production payment or identity verification.

1. Introduction

1.1 Project overview

HotelPro provides one interface for browsing suites, submitting a reservation, managing a stay, and carrying out day-to-day hotel operations. The client is a single-page React application. It communicates with a Node.js/Express REST API, which reads and writes data through Mongoose when MongoDB is connected.

1.2 Objectives

- Present available rooms with search, filtering, sorting, pricing, and amenity details.
- Support guest registration, sign-in, profile preferences, reservations, and stay history.
- Provide operational screens for check-in/check-out, housekeeping status, and guest service requests.
- Allow hotel administrators to manage room inventory, user roles, hotel settings, and analytics.
- Keep the application demonstrable during local development when a MongoDB service is not available.

1.3 Scope

The repository includes the client, backend API, database models, seed data, email helper, payment endpoints, and a backend API test script. It does not include a separate native mobile application. Several integrations have development-mode or simulated behavior; production deployment requires reviewing and configuring those integrations as outlined in Section 10.

2. Requirements

2.1 Functional requirements

Area	Requirement
Room discovery	List suites and filter by category, price range, guest capacity, and availability; search and sort results.
Room details	Display room information, amenities, pricing, images, and reviews.
Authentication	Register and sign in users; retrieve and update a guest profile; distinguish guest, staff, and admin roles.

Area	Requirement
Reservations	Create reservations, view permitted bookings, inspect an active stay, cancel a reservation, and perform staff check-in/check-out actions.
Hotel operations	Update room housekeeping status and seasonal pricing; view availability; create and update service requests.
Guest feedback	Read reviews and submit a review.
Payments and email	Request a payment intent and send reservation-related email when SMTP is configured.
Administration	View analytics, manage user roles, inspect audit-log data, and read/update hotel configuration.
System status	Expose a health endpoint that reports service and database connection status.

2.2 Non-functional requirements

- **Usability:** Provide responsive, role-oriented screens for guests, hotel staff, and administrators.
- **Maintainability:** Separate the client, API routes, controllers, models, middleware, and utilities.
- **Interoperability:** Use JSON over HTTP for client/API communication.
- **Security:** Hash database-backed passwords, use bearer JWTs for protected API routes, and restrict selected operations by role.
- **Availability for development:** Continue with in-memory sample data when MongoDB cannot be reached. This fallback is process-local and is not durable storage.
- **Configuration:** Keep runtime settings and integration credentials in environment variables rather than source-controlled secrets.

3. Software and Hardware Requirements

3.1 Software

Software	Purpose
Node.js and npm	Run the Express server, install JavaScript dependencies, and run project scripts.
MongoDB (optional for local demo)	Persistent document database. If it is unavailable, the server uses its in-memory fallback.
Modern web browser	Access the Vite-hosted React application.
Git	Clone and manage the project source.

The project is organized as a client and server package. Frontend dependencies include React 19, React Router, Redux Toolkit, Axios, Vite, Bootstrap, Material UI, Recharts, and React Toastify. The backend uses Express, Mongoose, JSON Web Tokens, bcryptjs, Stripe, Multer, and Nodemailer. Refer to the package manifests for exact dependency versions.

3.2 Suggested local development hardware

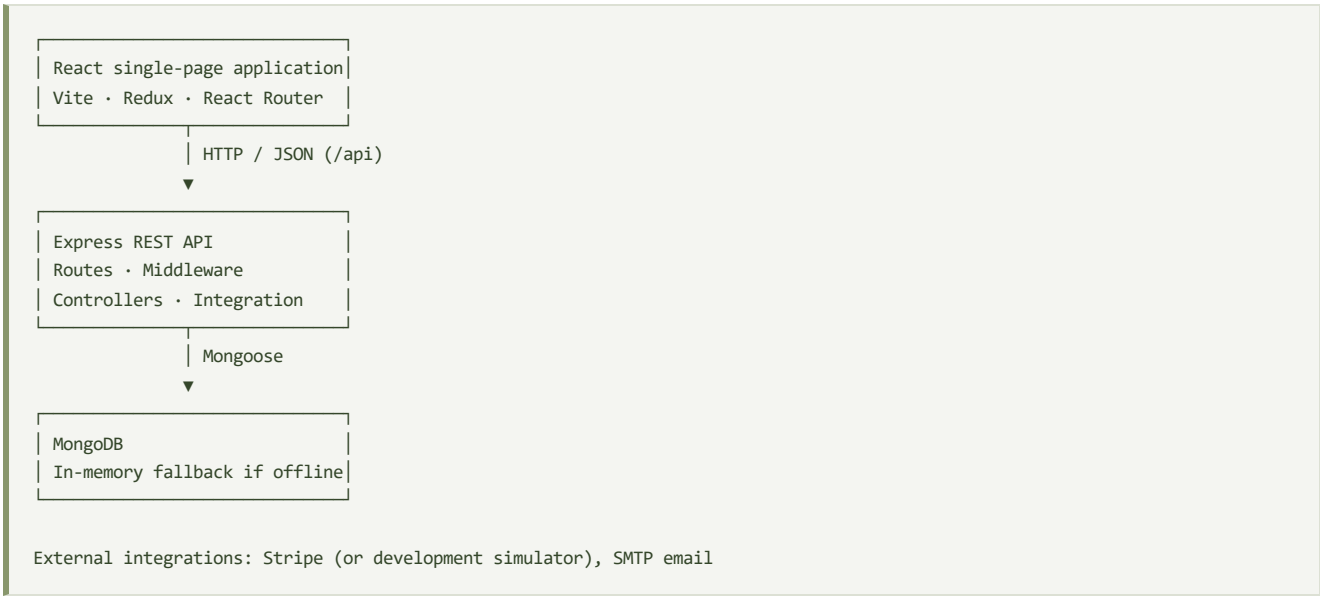
The following is a practical development recommendation, not an enforced project requirement:

- 64-bit dual-core processor or better

- 4 GB RAM or more
- Approximately 1 GB of free disk space for the source and installed dependencies
- Network access if installing packages or using remote integrations

4. System Architecture

4.1 Architecture overview



In development, Vite serves the client on port `5173` and proxies `/api` requests to the Express server on port `5000`.

4.2 Backend request flow

1. A client request reaches an Express router mounted under `/api`.
2. Protected routes use authentication middleware to validate a bearer token; selected routes additionally check the user's role.
3. A controller validates or interprets request data and applies the relevant business operation.
4. The controller reads or updates MongoDB through a Mongoose model, or uses the in-memory fallback where implemented.
5. The API returns a JSON response, normally with a `success` flag and result data.

4.3 Client structure

- `client/src/pages/` contains the main page-level experiences.
- `client/src/components/` contains shared navigation, modal, notification, and role-demo components.
- `client/src/redux/` contains application state slices and the Redux store.
- `client/src/services/api.js` defines the Axios client and API functions.

4.4 Server structure

- `server/routes/` maps HTTP paths to controller functions.

- `server/controllers/` implements API behavior.
- `server/models/` defines Mongoose schemas.
- `server/middleware/` provides authentication, error handling, and upload handling.
- `server/config/db.js` connects to MongoDB and reports database status.
- `server/utils/` contains seed data and email functionality.

5. Application Modules

5.1 Guest and public experience

The home, room listing, room detail, booking checkout, guest dashboard, and notifications pages support room discovery and guest interactions. Room search supports categories, price limits, capacity, availability, free-text search, and sorting. Profile data includes optional VIP preferences.

5.2 Reservation workflow

Reservations capture guest and room details, dates, number of nights, guest count, payment details, and special requests. The booking API supports listing bookings, retrieving a booking, viewing the active stay, cancellation, and staff check-in/check-out.

5.3 Staff operations

The staff dashboard provides operational views for arrivals and departures, room status, and service requests. The room status API supports housekeeping updates, and staff can progress service tickets through their statuses.

5.4 Inventory and hotel management

The hotel management page supports room inventory tasks. Room records include the room number, category/type, capacity, nightly pricing, amenities, images, availability, and housekeeping status. Staff and administrators can access the availability matrix; seasonal pricing updates are restricted to administrators.

5.5 Administration and analytics

The admin dashboard presents KPIs and chart data, user and role management, audit-log data, and hotel configuration. Configuration includes hotel contact information, tax, resort fees, currency, maintenance mode, and seasonal surge settings.

5.6 Reviews, services, and notifications

The API stores room reviews and service tickets. Notification data has a recipient role, category, read state, title, and message. The frontend also includes review, service-request, invoice, and notification components.

6. Data Design

The primary Mongoose models are:

Model	Main data
`User`	Name, unique email, hashed password, role (`guest`, `staff`, or `admin`), phone, loyalty tier, and VIP preferences.

Model	Main data
`Room`	Room number, title/type, prices, capacity, bed and size information, amenities, images, rating, availability, and housekeeping status.
`Booking`	Booking reference, guest and room details, stay dates, amounts, payment and booking statuses, special requests, and digital-key flag.
`Service`	Guest and room, service type, details, priority, status, and assignee.
`Review`	Room reference, guest name, rating, title, comment, and stay-verification flag.
`HotelConfig`	Hotel identity and contact details, tax rate, resort fee, currency, maintenance mode, and seasonal surges.
`Notification`	Recipient role, title, message, category, and read state.

MongoDB schemas use timestamps for applicable records. Seed data is defined in `server/utils/seedData.js`. When MongoDB is unavailable, the backend uses in-memory stores for supported entities; such changes are lost when that server process stops.

7. API Reference

All paths below are relative to the API base URL, `http://localhost:5000/api`. Protected routes require an `Authorization: Bearer <token>` header. Routes marked **Staff/Admin** or **Admin** additionally require the stated role.

7.1 Health and authentication

Method	Path	Access	Purpose
`GET`	`/health`	Public	Service and database status.
`POST`	`/auth/register`	Public	Register a guest account and return an authentication token.
`POST`	`/auth/login`	Public	Authenticate and return a token.
`GET`	`/auth/me`	Authenticated	Retrieve the current profile.
`PUT`	`/auth/profile`	Authenticated	Update profile information and preferences.

7.2 Rooms

Method	Path	Access	Purpose
`GET`	`/rooms`	Public	List rooms; supports `category`, `minPrice`, `maxPrice`, `capacity`, `availableOnly`, `search`, and `sort` query parameters.
`GET`	`/rooms/availability/matrix`	Staff/Admin	Retrieve the room availability matrix.
`GET`	`/rooms/:id`	Public	Retrieve a room by ID or room number.
`POST`	`/rooms`	Staff/Admin	Create a room; accepts up to five uploaded images.
`PUT`	`/rooms/:id`	Staff/Admin	Update a room.
`PATCH`	`/rooms/:id/status`	Staff/Admin	Change housekeeping status.

Method	Path	Access	Purpose
PATCH	/rooms/pricing/multiplier	Admin	Update seasonal pricing multiplier.

7.3 Bookings

Method	Path	Access	Purpose
POST	/bookings	Public or authenticated	Create a reservation.
GET	/bookings	Authenticated	List bookings according to the authenticated role and optional filters.
GET	/bookings/active/stay	Authenticated	Retrieve the current guest's active stay.
GET	/bookings/:id	Authenticated	Retrieve a booking by ID/reference.
POST	/bookings/:id/cancel	Authenticated	Cancel a reservation.
POST	/bookings/:id/checkin	Staff/Admin	Check a guest in.
POST	/bookings/:id/checkout	Staff/Admin	Check a guest out.

7.4 Services and reviews

Method	Path	Access	Purpose
GET	/services	Authenticated	List service requests.
POST	/services	Authenticated	Create a service request.
PATCH	/services/:id/status	Staff/Admin	Update service status and assignment.
GET	/reviews	Public	List reviews; may be filtered by room.
POST	/reviews	Public	Submit a review.

7.5 Payments and administration

Method	Path	Access	Purpose
POST	/payments/create-payment-intent	Public or authenticated	Create a Stripe intent when configured, otherwise return a development simulated intent.
POST	/payments/verify	Public or authenticated	Return the current verification response; see the production-readiness note below.
GET	/admin/analytics	Admin	Retrieve dashboard KPIs and chart data.
GET	/admin/users	Admin	List users.
PATCH	/admin/users/:id/role	Admin	Change a user's role.
GET	/admin/audit-logs	Admin	Retrieve audit-log data.
GET	/admin/config	Public	Read hotel configuration.
PUT	/admin/config	Admin	Update hotel configuration.

8. Installation and Operation

8.1 Prerequisites

Install Node.js with npm. MongoDB is optional for a local demonstration because the server has an in-memory fallback; use MongoDB when persistent data is required.

8.2 Install dependencies

From the repository root:

```
npm install --prefix server
npm install --prefix client
```

8.3 Configure the backend

Copy `server/.env.example` to `server/.env` and set values appropriate for the environment. The example includes:

```
PORT=5000
NODE_ENV=development
MONGO_URI=mongodb://127.0.0.1:27017/hotelpro
JWT_SECRET=replace_with_a_long_random_secret
JWT_EXPIRE=30d
STRIPE_SECRET_KEY=your_stripe_secret_key
SMTP_HOST=your_smtp_host
SMTP_PORT=2525
SMTP_USER=your_smtp_username
SMTP_PASS=your_smtp_password
EMAIL_FROM="HotelPro <your-sender@example.com>"
CLIENT_URL=http://localhost:5173
```

Use real credentials only in an untracked local `.env` or a secure deployment secret store. Do not use development fallback secrets or simulated payment responses in production.

8.4 Start the application

Open two terminals at the repository root:

```
# Terminal 1: API server
npm run server
```

```
# Terminal 2: Vite client
npm run dev
```

Open `http://localhost:5173`. The API is available at `http://localhost:5000/api`, and its health endpoint is `http://localhost:5000/api/health`.

8.5 Useful scripts

Command	Purpose
`npm run dev`	Start the client development server.
`npm run server`	Start the backend server.
`npm run client:build`	Build the frontend for production.
`npm run test`	Run the backend API test script against a server listening on `localhost:5000`.
`npm run seed --prefix server`	Run the backend seed utility.

The API test script does not start the server itself. Start the backend first, then run `npm run test` from the repository root.

9. Testing

The repository includes a 20-case backend API integration script at `server/tests/api.test.js`. It checks the health endpoint, room listing and filtering, authentication, booking operations, service requests, reviews, payment-intent response, and administrative endpoints.

Run the backend and then execute:

```
npm run test
```

The test script sends HTTP requests to `localhost:5000`; it therefore requires the API server to be running. Test results are dependent on the configured database or in-memory fallback and any required environment configuration.

10. Implementation Notes and Production Readiness

The project includes explicit development/demo behavior. Keep the following distinctions in mind when deploying or using the system for real guest transactions:

- **In-memory fallback:** It is useful for demonstrations but is volatile, local to one process, and unsuitable for durable or multi-instance production data.
- **Payment verification:** Without a live Stripe secret, payment-intent creation returns a simulated intent. The current `/payments/verify` handler returns a success-shaped response without checking settlement with Stripe. It must not be treated as proof of payment in a real booking flow.
- **Booking payment state:** Reservation creation currently records a paid status in its booking payload. Production booking confirmation should be tied to verified payment status and server-side pricing rules.
- **Demo authentication:** The backend includes development demo-token behavior and fallback JWT settings. Replace and constrain these before production use; issue real secrets through secure configuration.
- **Email delivery:** Confirmation email delivery requires valid SMTP settings. Without them, successful booking creation does not guarantee email delivery.
- **Access-control review:** Review authorization on every route before deployment, including public reservation, review, payment, and hotel-configuration read endpoints.

- **Operational hardening:** Configure restrictive CORS, validate request data and pricing server-side, use production-safe error logging, and add deployment monitoring and backups.

These are implementation notes for accurate handoff and deployment planning; they do not change the current project behavior.

11. Conclusion

HotelPro provides a full-stack foundation for hotel room discovery, reservations, guest profiles, staff operations, and administrative management. The repository separates the React client from the Express API and Mongoose data models, and includes seed data and API tests to support local development. Its demo and fallback integrations make it convenient to showcase, while the production-readiness items above identify the additional work required before processing live payments or handling persistent hotel operations.

Appendix A. Project Layout

```
HotelPro/
├── client/
│   ├── src/
│   │   ├── components/
│   │   ├── pages/
│   │   ├── redux/
│   │   └── services/
│   └── vite.config.js
├── server/
│   ├── config/
│   ├── controllers/
│   ├── middleware/
│   ├── models/
│   ├── routes/
│   ├── tests/
│   └── utils/
├── package.json
└── HOTELPRO_PROJECT_DOCUMENTATION.md
```